UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

COS730 – Assignment Two Report

Nobuhle Mtshali u22526171

Due date: 14 May 2026

Task One – Baseline Implementation

The baseline implementation does not introduce any optimisations, it only reproduces the behaviour that's illustrated on the sequence diagram given. Each lifeline in the diagram becomes a java class in the baseline package with each method implemented as public method on the receiving classes. The implementation is in the following directory src/main/java/baseline for inspections.

The implementation of the baseline system can also be found on the following GitHub repo: https://github.com/ReituTheCompSciGirlie/COS730-Assignment2

Each java class corresponds to a lifeline from the baseline sequence diagram. The table below makes illustrates the mapping of each lifeline which corresponds to a java class.

Lifeline	Java class
Researcher	This is the main class, which is the actor in this case
UI	This class forwards submitResearchOutput() to the controller
SubmissionController	This holds direct references to the four collaborators
Validator	This returns a Boolean with no reason code
Database	This stores submissions, the reviewer pool and raw score log
reviewerManager	This one has an external getAvailableReviewers with two self-calls
Reviewer	This persists scores directly to the database
EvaluationManager	Initiates the calculateAverage, checkConsensus, and applyRules self calls
NotificationService	Separate notifyAcceptance, notifyRejection, and notifyRevision methods

Task 2 – Design Analysis

The baseline interaction model exhibits structural weaknesses when assessed against the GRASP responsibility assignment patterns and against broader behavioural modelling criteria of cohesion, control flow clarity, and locality of decision logic. The five issues identified are discussed below**.**

2.1 Redundant message calls

The sequence diagram provided contains messages whose presence is a function of design rather than of the problem domain. The ReviewerManager invokes filterConflicts and checkWorkload as two separate calls on the same reviewer list. The NotificationService exposes the notifyAcceptance, notifyRejection, and notifyRevision , as three separate methods whereby a single polymorphic call parametrised by the outcome would still do the same thing. These extra messages expose the intermediate state on object interfaces and make the design harder to evolve. Each method is a contract that future callers depend on which increases the cost of every later change.

2.2 Violations of responsibility assignment principles

The SubmissionController breaches the controller pattern. A controller must delegate the requests to the business object and collaborates with the business objects to jointly handle the actor request, however the SubmissionController selects which reviewers to assign and then kicks off the evaluation by name.

The Reviewer objects directly call the database.savescore() at the end of the submitScore().The reviewer is not the information expert, it does not own the score record the evaluationManger does and it should not be coupled to the subsystem at all. This violates the GRASP information expert and low coupling and propagates the database reference into a class that models an academic peer.

2.3 High coupling and low cohesion

The SubmissionController holds a direct reference to four classes the validator, database, ReviewerManger , and the EvaluationManager, and indirectly drives the Reviewer by looping over a list it received. Its cohesion is low, validating, selecting reviewers and triggering evaluation are responsibilities of four and not facets of one.

The database is referenced by three classes the submissionController to save the submission, the reviewerManager to fetch the reviewers and the Reviewer to save the scores. This creates a dependency on the database class, which means that any change to the database interface ripples through three layers of the design.

2.4 Inefficient control flow

When the researcher submits the call stack passes through the UI, submissionController, Validator, then the Databsase, the ReviewerManager, and the database again, then back to the ReviewerManager for two self-calls. Each transition creates an opportunity for the state to be misinterpreted, as the reviewer knows less about the original request than the sender did.

2.5 Poor handling of decision logic

The decision logic is fragmented only on the three classes. The validator returns a Boolean that loses every distinction between missing title, and a page count out of range, and the controller can only reply with a generic error message. The ReviewerManager makes the filtering decisions through two sperate self-calls. The EvaluationManager.applyRules() implements the reject,revise,accept decision as a chain of if-else statements and the diagram duplicates the same branches as a three alt fragment which produces three different notify messages.

As no single artefact captures the rules, therefore an accessor can not verify that they are complete or unambiguous.

Task 3 – Decision Table

This section extracts and formalises all the decision logic embedded into the intelligent submission review system sequence diagram into three separate decision tables. There are three major decision points: the submission format validation, reviewer eligibility filtering, and final evaluation outcome determination.

1. Submission Format Validation

This decision logic is obtained from the alt fragment immediately after the validator.validateFormate whereby an invalid error is returned to the UI if any of the submission formats are invalid and if valid it proceeds to the saveSubmission operation.

	1	2	3	4
C1: Submission format is valid	Y	Y	N	N
C2: Required fields are present	Y	N	Y	N
Rule Count	1	1	1	1
A1: Save submission to database	X			
A2: return validation error to UI(invalid format)		X		
A3: return validation error to UI(missing fields)			X	
A4: Return validation error to UI(both invalid)				X

2. Reviewer Eligibility Filtering

This decision logic is embedded inside the loop [assign reviewers] with the calls to filterConflicts and checkworkloads. This decision logic has three conditions that manage whether a reviewer may be assigned which are conflict of interest, workload capacity and domain expertise.

	1	2	3	4	5	6	7	8
C1: Reviewer has conflict of interest	Y	Y	Y	Y	N	N	N	N
C2: Reviewer workload is at capacity	Y	Y	N	N	Y	Y	N	N

C3: Reviewer expertise matches domain	Y	N	Y	N	Y	N	Y	N
Rule Count	1	1	1	1	1	1	1	1
A1: Exclude reviewer	X	X	X	X	X	X		
A2: Add reviewer to filtered list							X	
A3: Add reviewer to list								X

3. Final Evaluation Outcome determination

After all reviewers submit the scores, the EvaluationManager calls the calculateAverage, checkConsensus, and applyRules. The results get into an alt fragment which is accepted, rejected, and revised which are the actions.

	1	2	3	4	5
C1: Average score >= acceptance threshold	Y	Y	Y	Y	N
C2: Reviewer consensus reached	Y	Y	N	N	Y
C3: All submission rules pass	Y	N	Y	N	N
Rule Count	1	2	2	2	2
A1: Accepted	X				
A2: Revision		X	X	X	
A3: Rejected					X

How the decision tables improve clarity and maintainability

1. **Clarity**

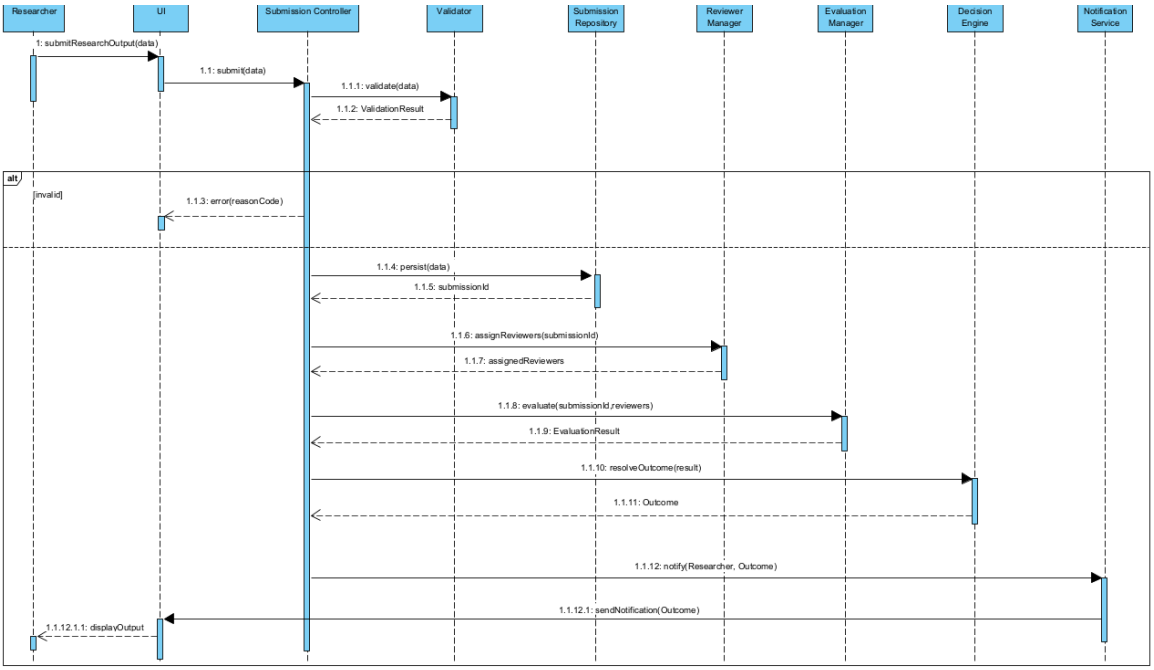
   The sequence table scatters decision logic across multiple loops and alt fragments which makes difficult to see all the conditions which result into a given outcome, the decision table however, presents all the conditions with their combinations in on place which makes all the cause effect relationships visible. The decision table eliminates ambiguity. With the sequence diagram there is no specification of what happens whenever a reviewer has no conflict and is not overloaded but lacks expertise, while the decision table ensures that this case is handled.

2. **Maintainability**

   With the sequence diagram when adding a new evaluation, it would require restricting nested alt fragments across multiple lifelines, whereas with the decision table the equivalent change would just require adding one row to the decision table and extending the rule count.

Task 4 – Sequence Diagram Optimisation

The optimised sequence diagram is still the same use case modelled with four design changes: the decision logic focuses on dedicated objects, responsibilities are assigned to their information experts, the database is replaced by the submissionRepository, and the redundant intermediate messages are removed.



Properties of new design

The optimised diagram shows higher cohesion where each object performs their responsibilities. Coupling is reduced whereby the submissionController interact with each collaborator through a single message that returns a value object. With this the submissionController would not have to know about the internal collections, loops or branching.

The control flow is now shallow therefore from the researcher to persist side effect there is one loop against the three from the baseline sequence.

Change by change rationale

In the optimised sequence diagram, the reviewerManager.assignReviewers replaces the getAvailableReviewers, filterConflicts,checkWorkload and the assign loop into one call.This follows the information expert and reduces the surface area of the submissionController.

The Reviewer no longer touches persistence, the reviewer,score(submission) now just returns a number and the evaluationManager persists the score through the submissionRepository.persistScore.

The EvaluationManager now returns an EvaluationResult, the three baseline self-calls become a single evaluate method which returns a value object. The NotificationService only uses only one notify method so the three notifyAcceptance, notifyRejection, and notifyRevision methods became one notify message dispatched on the outcome enum.

The database is now replaced by the submissionRepository, which is a pure fabrication that owns submissions, the reviewers pool and scores. Now the reviewer has no need to hold the database reference and the submissionController no longer calls a persistence interface directly.

Task 5 – Optimised Implementation

The optimised implementation is in the following directory src/main/java/optimised and reproduces the structure of the diagram from task 4. The implementation still has the same functional equivalence as the baseline implementation while applying the design changes mentioned above from the optimised sequence diagram.

SubmissionController

The submissionController is now the organiser: each step is now one message, each message returns a value object, and there are no loops, no lost manipulation, and no outcome-specific branches inside the controller.

```
28      public Outcome submit(Submission data) {
29          InteractionCounter.tick();
30          ValidationResult vr = validator.validate(data);
31          if (!vr.isValid()) return Outcome.INVALID_SUBMISSION;
32
33          repo.persist(data);
34          List<Reviewer> assigned = reviewerManager.assignReviewers(data);
35          EvaluationResult er = evaluationManager.evaluate(data, assigned);
36          Outcome outcome = decisionEngine.resolveOutcome(er);
37          notifier.notify(data.getAuthorEmail(), data, outcome);
38          return outcome;
39      }
40  }
```

This is the Controller pattern, which delegates tasks to its respective information expert, which returns the outcome to the method that called a specific operation.

- This controller does not consist of for loops, no subList, which were the features that the baseline controller has because the reviewer and notification were implemented incorrectly.

Validator

From the baseline implementation, the validator only returned a Boolean, which didn't specify which rule failed, the optimised validator returns a validationResult which indicates which rule failed

This change implements the GRASP recommendation which is that the validator should become the information expert for decision validity and the controllers coupling to the validator, which is reduced to just a single value returning call

```java
public class ValidationResult {
    public enum Reason { OK, MISSING_TITLE, BAD_EMAIL, EMPTY_CONTENT,
                         UNSUPPORTED_FORMAT, PAGE_OUT_OF_RANGE }

    private final boolean valid;
    private final Reason reason;

    private ValidationResult(boolean v, Reason r) { valid = v; reason = r; }

    public static ValidationResult ok() { return new ValidationResult(v: true, Reason.OK); }
    public static ValidationResult fail(Reason r) { return new ValidationResult(v: false, r); }

    public boolean isValid() { return valid; }
    public Reason getReason() { return reason; }
}
```

```java
public class Validator {
    public ValidationResult validate(Submission s) {
        InteractionCounter.tick();
        if (s == null || s.getTitle() == null || s.getTitle().isBlank())
            return ValidationResult.fail(ValidationResult.Reason.MISSING_TITLE);
        if (s.getAuthorEmail() == null || !s.getAuthorEmail().contains(s: "@"))
            return ValidationResult.fail(ValidationResult.Reason.BAD_EMAIL);
        if (s.getContent() == null || s.getContent().isBlank())
            return ValidationResult.fail(ValidationResult.Reason.EMPTY_CONTENT);
        String f = s.getFormat();
        if (f == null || !(f.equals(anObject: "PDF") || f.equals(anObject: "DOCX")))
            return ValidationResult.fail(ValidationResult.Reason.UNSUPPORTED_FORMAT);
        if (s.getPageCount() <= 0 || s.getPageCount() > 50)
            return ValidationResult.fail(ValidationResult.Reason.PAGE_OUT_OF_RANGE);
        return ValidationResult.ok();
    }
}
```

ReviewerManager

The filterConflicts and checkworkload methods from the baseline were changed to a single linear scan over the candidate pool therefore, the ReviewerManager now performs the assignment, so the submissionController no longer manipulates the reviewers' state. This change ensures that the expert pattern is not violated as the ReviewerManager is the class that has all the information needed to decide on who reviews which submission.

```java
    public List<Reviewer> assignReviewers(Submission s) {
        InteractionCounter.tick();
        List<Reviewer> pool = repo.reviewerPool();
        List<Reviewer> chosen = new ArrayList<>(TARGET_REVIEWERS);
        // Single linear pass: filter and select in one sweep
        for (Reviewer r : pool) {
            if (chosen.size() >= TARGET_REVIEWERS) break;
            if (r.getCurrentWorkload() >= MAX_WORKLOAD) continue;
            if (r.getConflictsWith().contains(s.getAuthorEmail())) continue;
            r.incrementWorkload();
            chosen.add(r);
        }
        return chosen;
    }
```

.

SubmissionRespository

This class replaces the database from the baseline implementation, it is a pure fabrication which bundles persistence operations into a single class and does not model a domain concept.

In the baseline implementation, three classes held references to the database; in the optimised system, there are only two: the ReviewerManager and the evaluationManager.

```
public class SubmissionRepository {
    private final Map<String, Submission> submissions = new HashMap<>();
    private final List<Reviewer> reviewerPool = new ArrayList<>();
    private final Map<String, List<int[]>> scores = new HashMap<>();

    public void seedReviewer(Reviewer r) { reviewerPool.add(r); }

    public String persist(Submission s) {
        InteractionCounter.tick();
        submissions.put(s.getId(), s);
        return s.getId();
    }

    public List<Reviewer> reviewerPool() {
        InteractionCounter.tick();
        return new ArrayList<>(reviewerPool);
    }

    public void persistScore(String submissionId, String reviewerId, int score) {
        InteractionCounter.tick();
        scores.computeIfAbsent(submissionId, k -> new ArrayList<>())
              .add(new int[]{reviewerId.hashCode(), score});
    }

    public int countScores(String submissionId) {
        return scores.getOrDefault(submissionId, new ArrayList<>()).size();
    }
}
```

NotificationService

The optimised version has only one method that returns the outcome, whereas the baseline implementation had three notification methods that required the EvaluationManager to switch the outcome to call the right one.

Therefore, whenever you add a new outcome value it doesn't change the notification service which applies the polymorphism pattern. The variation points are not duplicated as separate methods but are parameterised.

```java
public class NotificationService {
    private final List<String> log = new ArrayList<>();

    public void notify(String recipient, Submission s, Outcome outcome) {
        InteractionCounter.tick();
        log.add(outcome.name() + ":" + s.getId() + "->" + recipient);
    }

    public List<String> getLog() { return log; }
}
```

Task 6 – Empirical Evaluation and Comparison

This section performs empirical evaluations and comparisons of both the baseline system and the optimised system. The systems are compared based on the following metrics: number of method calls, execution time, code complexity, and maintainability indicators.

Number of method calls

The number of inter-object messages exchanged during a single submission is used to measure the architectural improvement. Fewer messages mean less coupling, lower overhead and fewer points of failure.

Quantity	Baseline	Optimised	Change
Interactions per submission	14	9	-35.7%
Interactions per 50000 runs(millions)	0,70	0,45	-35.7%

Execution Time

The execution time was measured against five trials of 20000 submissions after the 2000 iteration. For the optimised system, the execution time for each submission had a mean of 1175 ns, a 20,5% reduction from 1478 ns obtained from the baseline system.

Quantity	Baseline	Optimised	Change
Mean per submission time	1478	1175	-20.5%
Standard deviation across trials	359	262	
Throughput	676581	850841	25,8%

Maintainability Indicators

1. Quantitative Indicators

The optimised system accommodates better maintainability, so whenever an outcome category is added to the system, only a few classes would need to be changed compared to the baseline system. Compared to the baseline system, the changes that occur in the system are more about additions rather than invasions; therefore, there are no changes to the existing rules.

Artefact that must change	Baseline	Optimised
EvaluationManager.applyRules	Yes	No

EvaluationManager.startEvaluation	Yes	No
NotificationService	Yes	No
Sequence diagram – new alt arm	Yes	No
DecisionEngine.Rules	No	Yes
Outcome enum	No	Yes
Total of changes	4	2

When testing outcome resolution logic on the baseline system, it deals with the evaluationManager.applyRules reading files that are populated by other methods.This includes constructing a full EvaluationManager which needs to have a NotificationService and a list of Reviewer instances, each of which holds a database reference which involves 5 classes compared to the optimised system, which only include 2 classes for testing the outcome resolution logic.

2. Qualitative Indicators

Every GRASP pattern that the baseline system violates the optimised system ensures that it is honoured.

Pattern	Baseline	Optimised
Expert	The Reviewer.saveScore violates this pattern as the reviewer is not the expert on persistence	The EvaluationManger honours the following pattern as it owns the scoring and persistence
Low Coupling	The submissionController depends on 4 collaborators which violates low coupling	The submissionController depends on the 6 collaborators but this is only through a single value returning call.
High Cohesion	The EvaluationManger mixes the scoring ,rules, and evaluation which violates high cohesion being maintained in the system	In this system the responsibilities are split depending on the classes. The EvaluationManger deals with scoring and the decisionEngine decides and the NotificationService notifies
Polymorphism	With this system there are three different notify methods for each outcome which violates polymorphism	This system has only one notify method which handles all outcomes

Pure Fabrication	The database handles all classes including the Reviewer class	The submissionRepository introduced only deals with persistence only
Indirection	The decision logic is embedded directly on the applyRules method	For this system the decisionEngine acts as a indirection layer between results and outcomes

Justification of Improvements

1. The interactions per submission decreased from 14 to 9. Combining the two reviewer filters to one message, eliminating the score persistence from the Reviewer eliminated three methods which reduced the three EvaluationManager self calls into one evaluate method and a polymorphic notify method was implemented which replaced the three notify methods. All these changes are justified with the GRASP patterns.
2. The execution time decreased from 1478 to 1175 . The reduction is slight larger than the interaction count change because the smaller methods introduced in the optimised system inline better with the one pass evaluate method and computes its three statistics in one loop.
3. Adding a new outcome rule now allows for edits on only two files which are the outcome enum and the decisionEngine rule rather than the four from the baseline system which improves the maintainability of the system. Secondly, the outcome logic is now easier to test since there is only two files rather than five.

All these improvements share the same cause, which is the GRASP-driven redistribution of responsibilities. Since each class is the expert for the data it operates on, this means that low coupling and high cohesion follow and ensuring that there is no duplication of variation points, polymorphism follows.

Trade-offs introduced

Although the optimised system improved and ensured that GRASP patterns are not violated, there are a few trade-offs that it introduced, which are:

More files to navigate – The optimised system has more classes compared to the baseline system, therefore the developer when opening the project for the first time would just see so many files.

More direct collaborators on submissionController – Compared to the baseline system the optimised system has 6 collaborations, whereby each collaboration is a single call returning a value object.

Indirection in DecisionEngine – The four anonymous rule instances in the decision table add one layer of indirection, which could not be done but a simple if-else chain. Therefore, adding a new outcome does not affect any class since it only requires appending one entry to the rule list.